

『生无所息，斗无所止』



HUST Artificial Intelligence Robot laboratory

哈尔滨理工大学AIR人工智能机器人创新实验室

电控组第五讲

培训时间：第 10 周 11.7 主讲人：_____

关于时间安排：



周次与日期	培训内容	本节重点	课后成果
军训培训 9月25日	C语言①：环境配置与基础语法	编译运行过程、变量、数据类型、常量、运算符、输入输出、类型转换	能独立创建工程并完成简单计算程序
第7周 10月17日	C语言②：程序流程与函数，数组、字符串、结构体与指针	if、switch、for、while、函数、参数、返回值、作用域、数组下标、字符串结尾、结构体组织数据、地址与指针、指针传参	掌握分支语句、结构体、指针等高阶语句，初步掌握C语言
第8周 10月24日	STM32①：GPIO、按键与外部中断	工程建立、GPIO输入输出、LED、上下拉、EXTI、按键消抖	完成按键控制LED，并能处理按键抖动
第9周 10月31日	STM32②：定时器、PWM	定时器周期计算、非阻塞闪灯、PWM频率与占空比、舵机脉宽控制	完成PWM控制LED呼吸灯与舵机角度
第10周 11月7日	STM32③：UART与I ² C OLED	串口收发、字符串转数字、命令解析；OLED初始化和数据显示	串口控制频率/占空比/舵机角度，OLED显示系统状态
第11周 11月14日	正式考核	独立完成、录制视频演示、代码内容讲解	在安排的规定考核时段内完成，周日前提交



目录

01

串口基础

02

串口收发与中断

03

命令解析

04

OLEΔ 显示

『生无所息，斗无所止』



HUST Artificial Intelligence Robot laboratory

哈尔滨理工大学AIR人工智能机器人创新实验室

01 串口基础



1. 三根线：TX、RX、GND

我方的发（TX）接对方的收（RX），交叉接线；地线必须共在一起，否则收到的全是乱码。

2. 波特率：两边必须一样

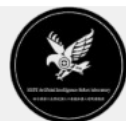
常用 115200。串口没有时钟线，全靠两边约定同一个速度，差一点就是满屏乱码。

3. CubeMX 里怎么配

选 Asynchronous 模式，8 位数据、无校验、1 位停止位；引脚会自动分配好。

```
// CubeMX 里 USART1 的配置
// Mode : Asynchronous
// Baud Rate : 115200
// Word Length : 8 Bits
// Parity : None
// Stop Bits : 1
// 引脚自动分配 PA9 / PA10
```

串口没有时钟线，两边全靠约定：线要交叉、地要共、波特率必须一致。



1. 发送就是一句调用

HAL_UART_Transmit 把一段字节发出去，长度和超时要自己写清楚，发完才返回。

2. 数字要先变成字符

串口只能发字节，直接发一个 int 会得到乱码。用 sprintf 拼成字符串再发。

3. 换行别忘

字符串结尾加上回车换行，串口助手才一行一行显示，看着才清楚。

```
char buf[32];
sprintf(buf, "duty=%d\r\n", duty);
HAL_UART_Transmit(&huart1,
(uint8_t *)buf, strlen(buf), 100);
// 长度用 strlen，别写死
// 超时 100ms，够用了
```

发送是主动的、阻塞的：先转成字符，再写清长度，最后记得换行。

『生无所息，斗无所止』



HUST Artificial Intelligence Robot laboratory

哈尔滨理工大学AIR人工智能机器人创新实验室

02 串口收发与中断



1. 别在主循环里等

用接收中断：开一个字节的接收，收到就进回调，主循环完全不用守着串口。

2. 一个字节一个字节攒

回调里把字节存进缓冲区，遇到换行就认为一句话结束，再加一个结束符。

3. 收完必须再开一次

回调末尾要重新调用接收函数，不然只收得到第一个字节，后面全部丢失。

```
uint8_t rb, buf[64], len = 0;
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *h)
{
    buf[len++] = rb;
    if (rb == '\n') { buf[len - 1] = 0; len = 0; }
    HAL_UART_Receive_IT(&huart1, &rb, 1);
}
// main 里先开一次
```

接收靠中断：开一个字节、收一个攒一个、遇到换行算一句、收完再开下一次。



1. 字符和数字不是一回事

收到的是 '5' 这个字符，它的值是 53。要参与计算，必须先转成真正的数字 5。

2. 用 atoi 或 sscanf

atoi("500") 直接得到 500；sscanf 还能按格式一次拆出多个数字。

3. 先看命令头，再取值

命令是「字母 + 数字」的格式，第一个字符决定做什么，后面的部分才是参数。

```
// 收到 "P500"
if (buf[0] == 'P') {
    int v = atoi((char *)&buf[1]); // 500
    if (v <= 999) {
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2, v);
    }
}
```

串口里只有字符没有数字：先转换、再限幅，最后才交给硬件用。

『生无所息，斗无所止』



HUST Artificial Intelligence Robot laboratory

哈尔滨理工大学AIR人工智能机器人创新实验室

03 命令解析



1. 先定一套命令表

写代码前先把命令定下来：A 开灯、B 关灯、P 加数值改占空比、S 查询状态。

2. 一次只处理一条

用 switch 认第一个字符就够；处理完立刻清空缓冲区，避免上一条的残留影响下一条。

3. 每条命令都回一句

执行完回一个 OK 或当前状态，上位机才知道命令生效了，调起来省一半时间。

```
switch (buf[0]) {  
case 'A': LED_ON(); break;  
case 'B': LED_OFF(); break;  
case 'P': duty = atoi((char *)&buf[1]);  
PWM_SET(duty); break;  
}  
UART_SEND("OK\r\n");
```

命令表定清楚、解析只认第一个字符、每条都回话——串口就从调试口变成了控制口。

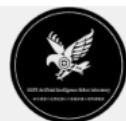
『生无所息，斗无所止』



HUST Artificial Intelligence Robot laboratory

哈尔滨理工大学AIR人工智能机器人创新实验室

0401EΔ 显示



1. 两根线：SCL 和 SDA

OLED 走 I2C，时钟线上拉后接 SCL，数据线接 SDA，供电接 3.3V；CubeMX 里开 I2C1 即可。

2. 初始化一次，然后清屏

上电只做一次初始化，接着清屏；之后就能按坐标往指定位置写字。

3. 显示什么：挑最关键的几项

灯的开关、当前占空比、舵机角度；数字先用 sprintf 转成字符串再显示。

```
OLED_Init();  
OLED_Clear();  
char line[20];  
sprintf(line, "LED:%s", on ? "ON " : "OFF");  
OLED_ShowString(0, 0, (uint8_t *)line, 16);  
sprintf(line, "duty=%d", duty);  
OLED_ShowString(0, 2, (uint8_t *)line, 16);
```

OLED 是你自己给自己的调试窗口：初始化一次，之后想写哪就写哪。



1. 串口点灯：发 A 灯亮回 1，发 B 灯灭回 0

先在 main 里轮询接收，把「收命令 → 执行 → 返回值」这条链路跑通。

2. 用接收中断重做任务 1

换成中断接收，回调里攒够一句再解析；注意处理缓冲区越界和收完再开。

3. OLED 显示小灯状态

把灯的开关状态显示在屏幕上，命令生效后屏幕要跟着变。

4. 课余：了解 UART 的其他用法

DMA 收发、空闲中断、printf 重定向、多串口；挑一两个试试，下一阶段会用到。

四题把串口收发、中断接收和 OLED 串成一套；第 11 周考核里这几样都要用上。



1. 代码注释必须统一 UTF-8 ；交上来打不开、或者注释是乱码的，按未提交处理。
2. 演示视频、代码和其他材料放在同一个压缩包里，不要拆成几个文件分开发。
3. 压缩包命名格式：班级 + 姓名 + 第几次任务，例如「电气 26-5 班 + 李华 + 第一次任务.zip」。
4. 发到指定邮箱，邮件主题与压缩包同名，压缩包作为附件上传，不要用网盘链接。
5. 不强制每周内完成，但转入梯队考核上传结束时间前必须全部交齐（第 11 周）。

五条要求都是硬性的：代码写得对，但包名写错、注释乱码，一样按未提交处理。

『生无所息，斗无所止』



HUST Artificial Intelligence Robot laboratory

哈尔滨理工大学AIR人工智能机器人创新实验室

下课啦!